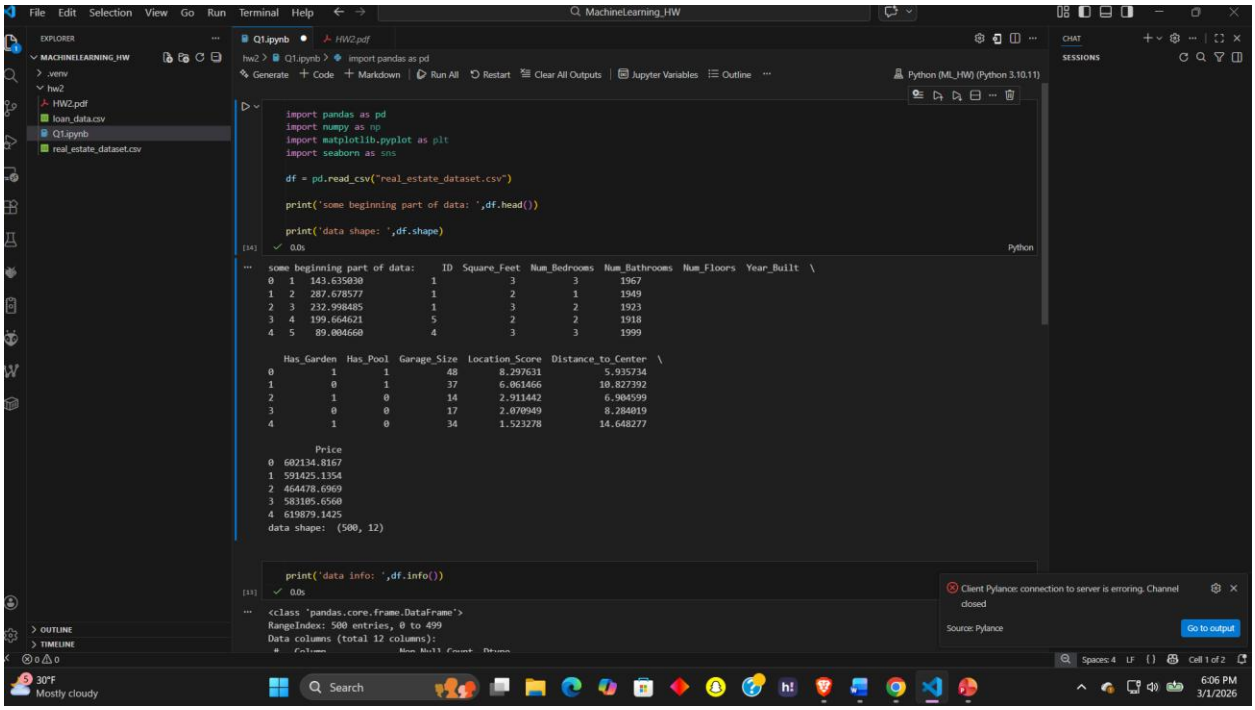
ECE 54810 – Machine Learning

Homework – Linear / Logistic Regression + Gradient Descent

1. Dataset Characteristics:
   a) number of samples: 500
      Input features: 11 input features
      , technically 10 input features, as we can exclude ID as there is no correlation between ID and price.
   b) The input features are:
      1. ID (We can exclude this)
      2. Square_Feet
      3. Num_Bedrooms
      4. Num_Bathrooms
      5. Num_Floors
      6. Year_Built
      7. Has_Garden
      8. Has_Pool
      9. Garage_Size
      10. Location_Score
      11. Distance_to_Center
2. Data Exploration and Visualization:
   a)



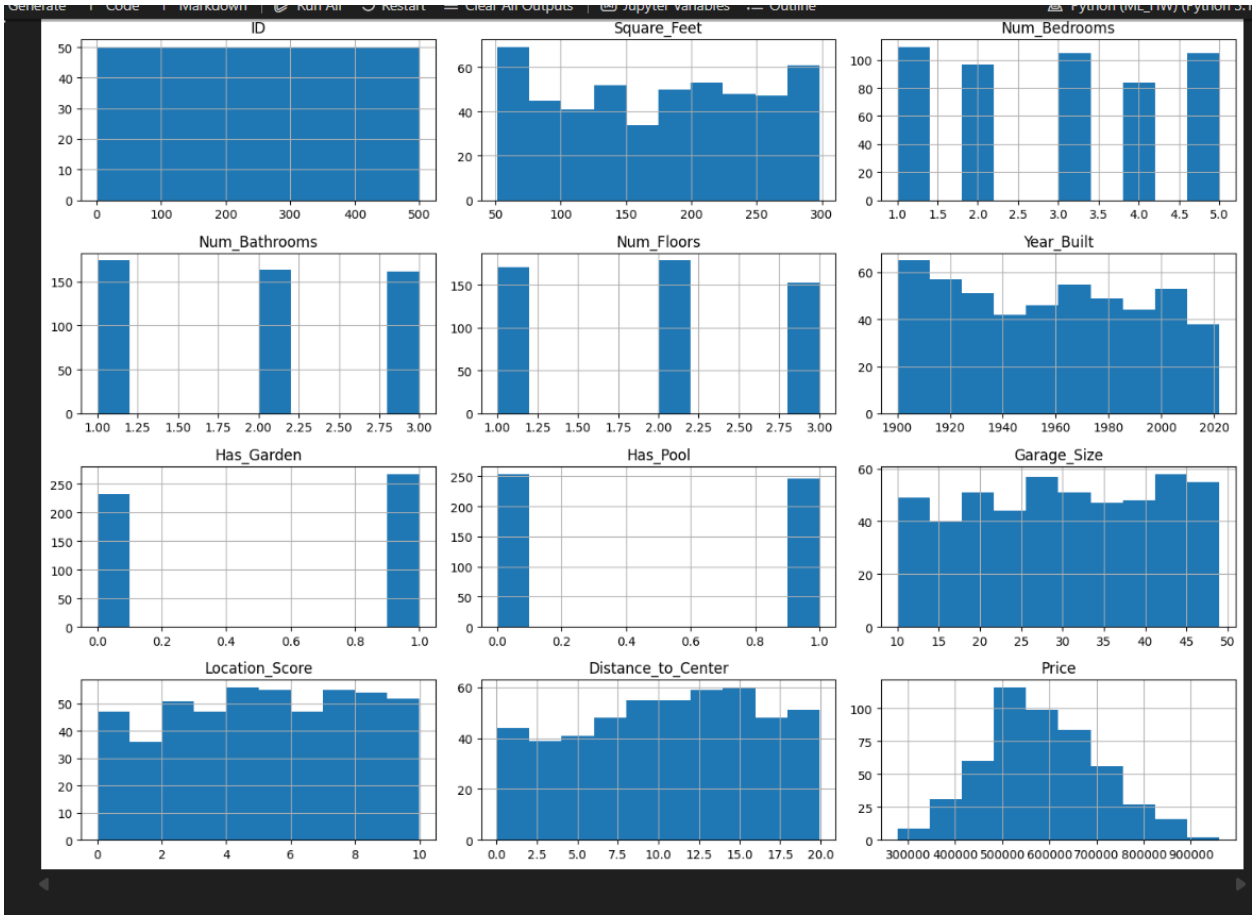
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 12 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   ID                  500 non-null    int64
 1   Square_Feet         500 non-null    float64
 2   Num_Bedrooms        500 non-null    int64
 3   Num_Bathrooms       500 non-null    int64
 4   Num_Floors          500 non-null    int64
 5   Year_Built          500 non-null    int64
 6   Has_Garden          500 non-null    int64
 7   Has_Pool            500 non-null    int64
 8   Garage_Size         500 non-null    int64
 9   Location_Score      500 non-null    float64
 10  Distance_to_Center  500 non-null    float64
 11  Price               500 non-null    float64
dtypes: float64(4), int64(8)
memory usage: 47.0 KB
data info:  None
```

As we see all the input features are integers or floats

We also got the mean, standard deviation, min and max using describe function.

```
                ID  Square_Feet  Num_Bedrooms  Num_Bathrooms  Num_Floors  \
count   500.000000   500.000000    500.000000     500.000000  500.000000
mean    250.500000   174.640428      2.958000       1.976000    1.964000
std     144.481833    74.672102      1.440968       0.820225    0.802491
min       1.000000    51.265396      1.000000       1.000000    1.000000
25%     125.750000   110.319923      2.000000       1.000000    1.000000
50%     250.500000   178.290937      3.000000       2.000000    2.000000
75%     375.250000   239.031220      4.000000       3.000000    3.000000
max     500.000000   298.241199      5.000000       3.000000    3.000000

         Year_Built  Has_Garden    Has_Pool  Garage_Size  Location_Score  \
count   500.000000  500.000000  500.000000   500.000000      500.000000
mean   1957.604000    0.536000    0.492000    30.174000        5.164410
std      35.491781    0.499202    0.500437    11.582575        2.853489
min    1900.000000    0.000000    0.000000    10.000000        0.004428
25%    1926.000000    0.000000    0.000000    20.000000        2.760650
50%    1959.000000    1.000000    0.000000    30.000000        5.206518
75%    1988.000000    1.000000    1.000000    41.000000        7.732933
max    2022.000000    1.000000    1.000000    49.000000        9.995439

       Distance_to_Center          Price
count          500.000000     500.000000
mean            10.469641  582209.629531
std              5.588197  122273.390347
min              0.062818  276892.470100
25%              6.066754  503080.344175
50%             10.886066  574724.113350
75%             15.072590  665942.301300
max             19.927966  960678.274300
```

Some histograms:

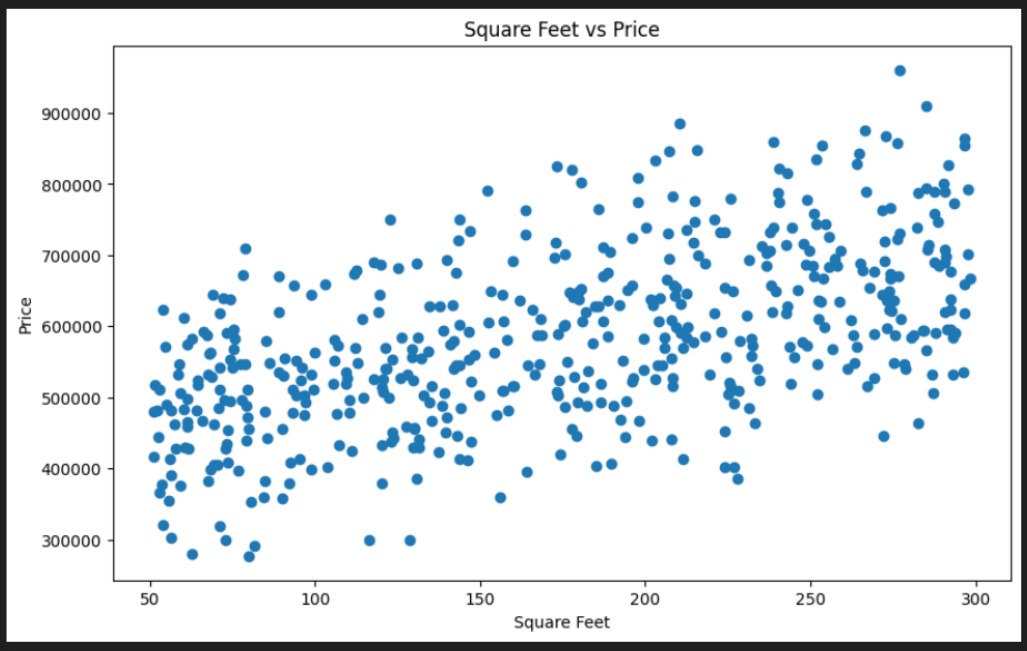


The histogram makes it even clearer that there is no relationship between ID and Price.

b) Plot of area vs price:

```
# Question 2.b: Scatter plot of Square_Feet vs Price
plt.figure(figsize=(10,6))
plt.scatter(df["Square_Feet"], df["Price"])
plt.xlabel("Square Feet")
plt.ylabel("Price")
plt.title("Square Feet vs Price")
plt.show()
```

0.1s



I also plotted some other features to get some more idea of distribution as well.

3. a) We will be excluding ID from our calculation, because there is no correlation between ID and house price.

```
# Question 3
# Normal equation: theta = (X^T * X)^(-1) * X^T * y
y = df["Price"].values
X = df.drop(columns=["ID","Price"]).values
# now adding the first column as 1,1,1,1... for the bias term
ones = np.ones((X.shape[0], 1))
X=np.hstack((ones, X))
theta = np.matmul(np.linalg.inv(np.matmul(np.transpose(X), X)), np.matmul(np.transpose(X), y))
theta
```

[39] 0.0s Python

```
array([-2.89662923e+06,  1.01780336e+03,  5.06851880e+04,  2.99479954e+04,
        2.11340195e+04,  1.51824128e+03,  3.04555023e+04,  4.74152538e+04,
        1.13527772e+03,  4.80536342e+03, -1.94489819e+03])
```

So our theta is:

[-2.89662923e+06, 1.01780336e+03, 5.06851880e+04, 2.99479954e+04,
2.11340195e+04, 1.51824128e+03, 3.04555023e+04, 4.74152538e+04,
1.13527772e+03, 4.80536342e+03, -1.94489819e+03]

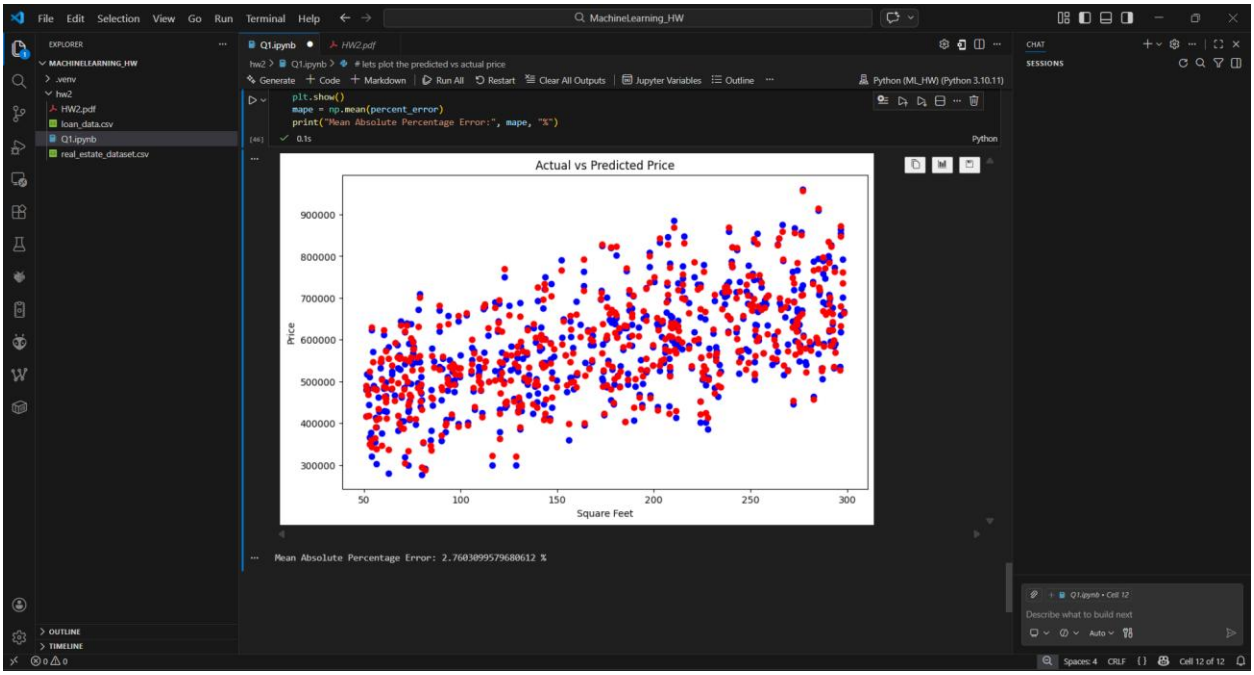
$\Theta 0$ = -2.89662923e+06
$\Theta 1$ = 1.01780336e+03
$\Theta 2$ = 5.06851880e+04
$\Theta 3$ = 2.99479954e+04
$\Theta 4$ = 2.11340195e+04
$\Theta 5$ = 1.51824128e+03
$\Theta 6$ = 3.04555023e+04
$\Theta 7$ = 4.74152538e+04
$\Theta 8$ = 1.13527772e+03
$\Theta 9$ = 4.80536342e+03
$\Theta 10$ = -1.94489819e+03

So our prediction model is :

Y(pred) = -2.89662923e+06 + 1.01780336e+03 * X1 + 5.06851880e+04*X2 + 2.99479954e+04*X3 + 2.11340195e+04*X4 + 1.51824128e+03*X5 + 3.04555023e+04*X6 + 4.74152538e+04*X7 + 1.13527772e+03*X8 + 4.80536342e+03*X9 + -1.94489819e+03*X10

Where,

X1 = Square_Feet
X2 = Num_Bedrooms
X3 = Num_Bathrooms
X4 = Num_Floors
X5 = Year_Built
X6 = Has_Garden
X7 = Has_Pool
X8 = Garage_Size
X9 = Location_Score
X10 = Distance_to_Center



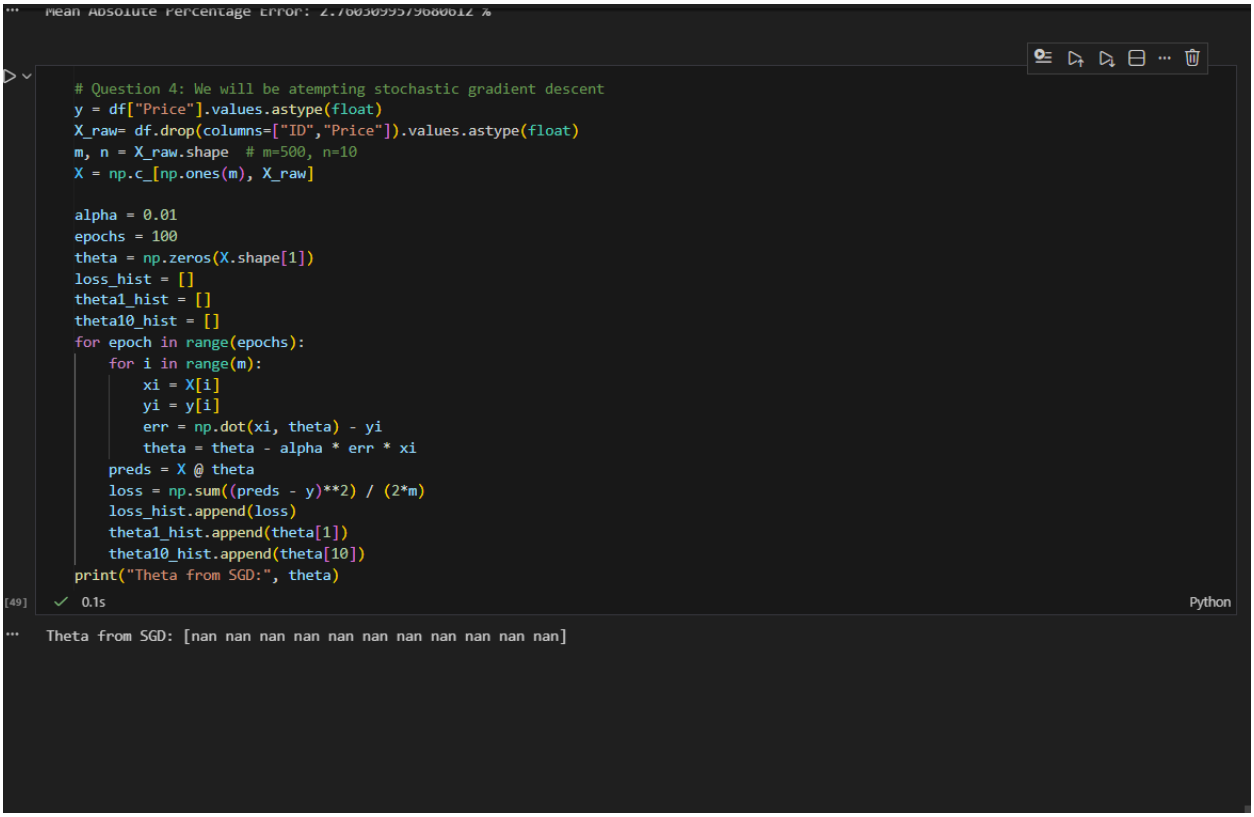
(I tried seeing the percentage error for square area with out prediction model, error percentage was 2.7)

4. a) We will be using stochastic gradient descent,

repeat
  for $i = 1$ to n **do**
    for every j **do**
      $\theta_j \leftarrow \theta_j + \alpha \sum_{i=1}^{n} \left(y^{(i)} - h_\theta(x^{(i)}) \right) x_j^{(i)}$
    end for
  end for
until convergence

Algorithm 1.3. Stochastic gradient descent.

We didn't normalize the data initially,

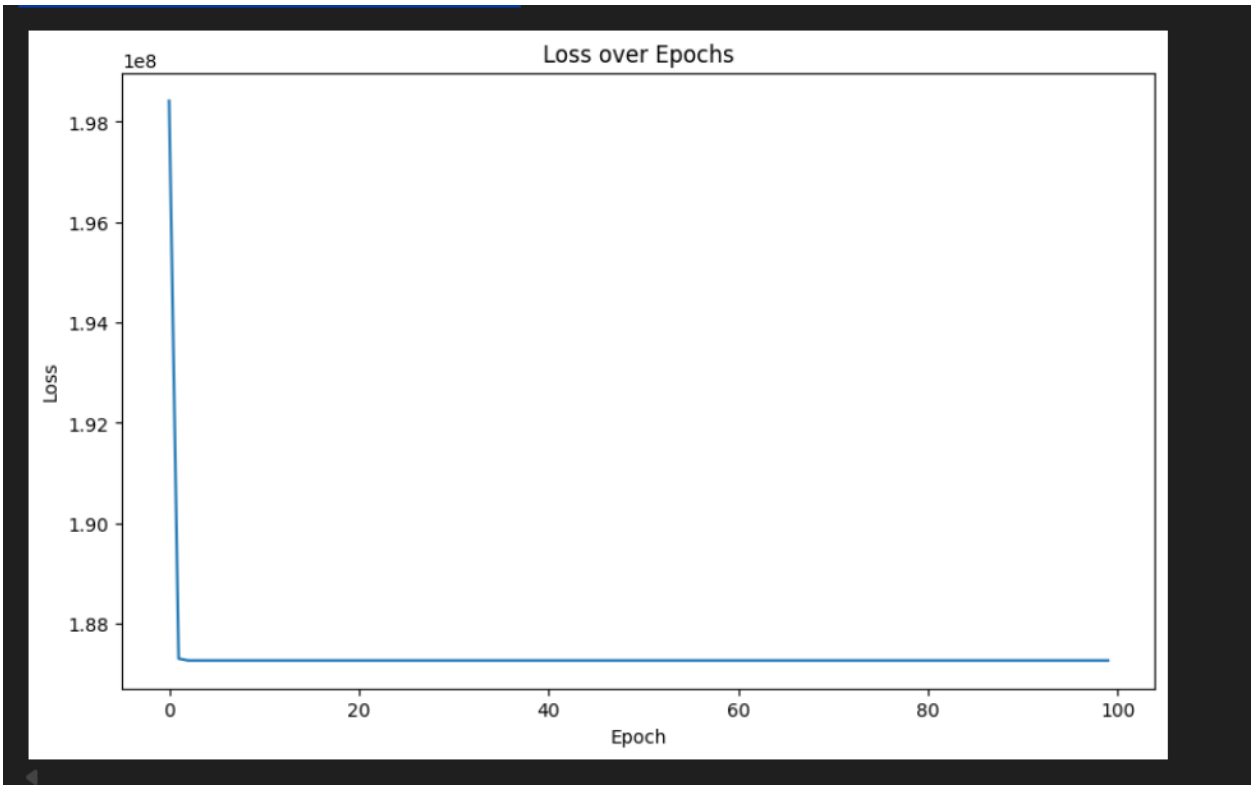


Probably happened because the numbers became too large, without normalization, the error number became too large.

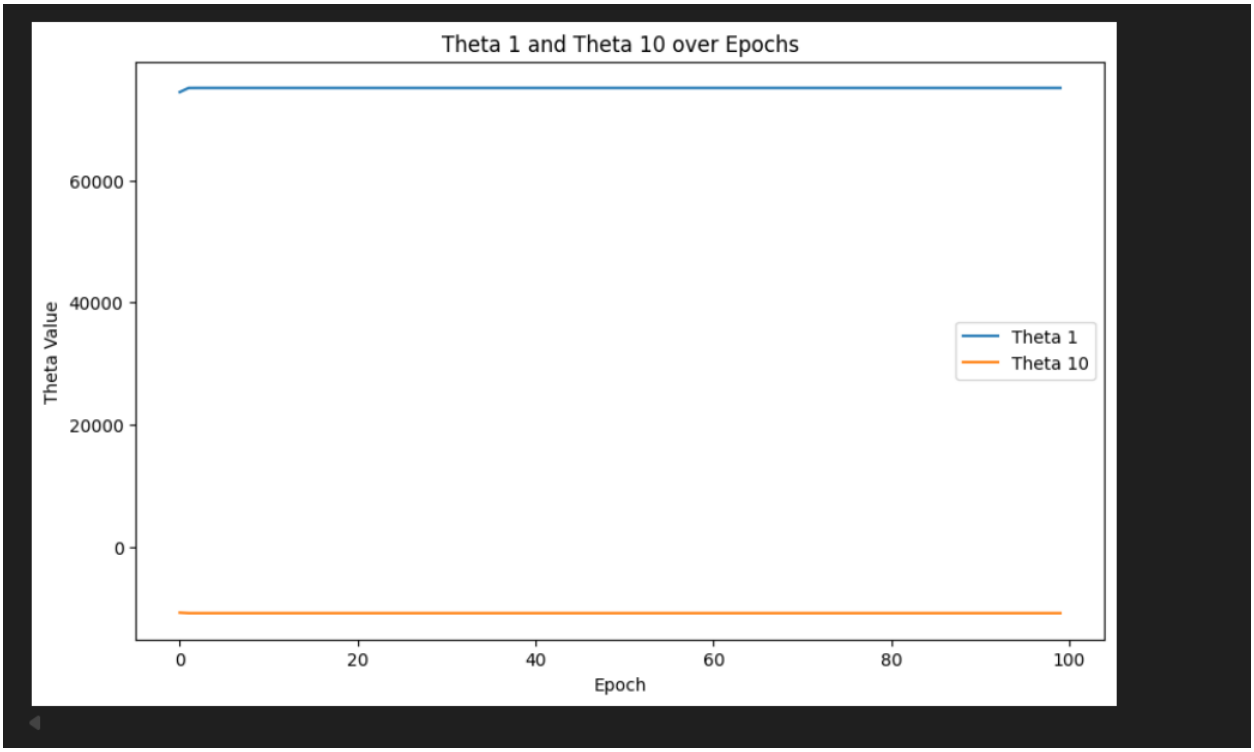
Now with normalization,

Theta from SGD: [580370.03743114 75118.58890982 73106.7874379 23371.3201633 16537.99838285 52491.98921087 17359.73187618 23387.80480013 11302.61188741 13099.9578661 -10884.58463814]

b) i)


Loss over Epochs
1e8
1.98
1.96
1.94
1.92
1.90
1.88
Loss
0
20
40
60
80
100
Epoch

ii)

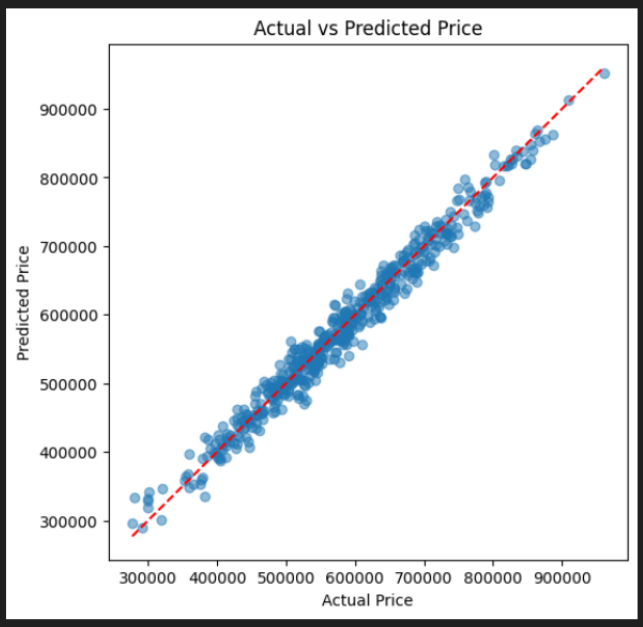

Theta 1 and Theta 10 over Epochs
60000
40000
20000
0
Theta Value
Theta 1
Theta 10
0
20
40
60
80
100
Epoch

Now,

I also tested the error for Stochastic gradient descent for square area,

```
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Price")
plt.show()
mape = np.mean(np.abs((y - y_pred) / y)) * 100
print("MAPE:", mape, "%")
```

[10] ✓ 0.1s Python



MAPE: 2.8046069673487586 %

The error is larger than using Normal Equation.

c) The loss function plot shows a rapid decrease during the first few epochs followed by a nearly constant value. Initially, the model parameters were far from optimal because all parameters were initialized to zero, causing large prediction errors. Consequently, the gradient magnitude was large and the algorithm quickly adjusted the parameters toward the optimal region

The parameter plots for θ_1 and θ_{10} also confirm convergence. Both parameters changed significantly during the early epochs as the model learned the relationship between the input features and house prices.

5. The parameter values obtained using the normal equation and stochastic gradient descent are not identical. The normal equation computes the exact optimal solution

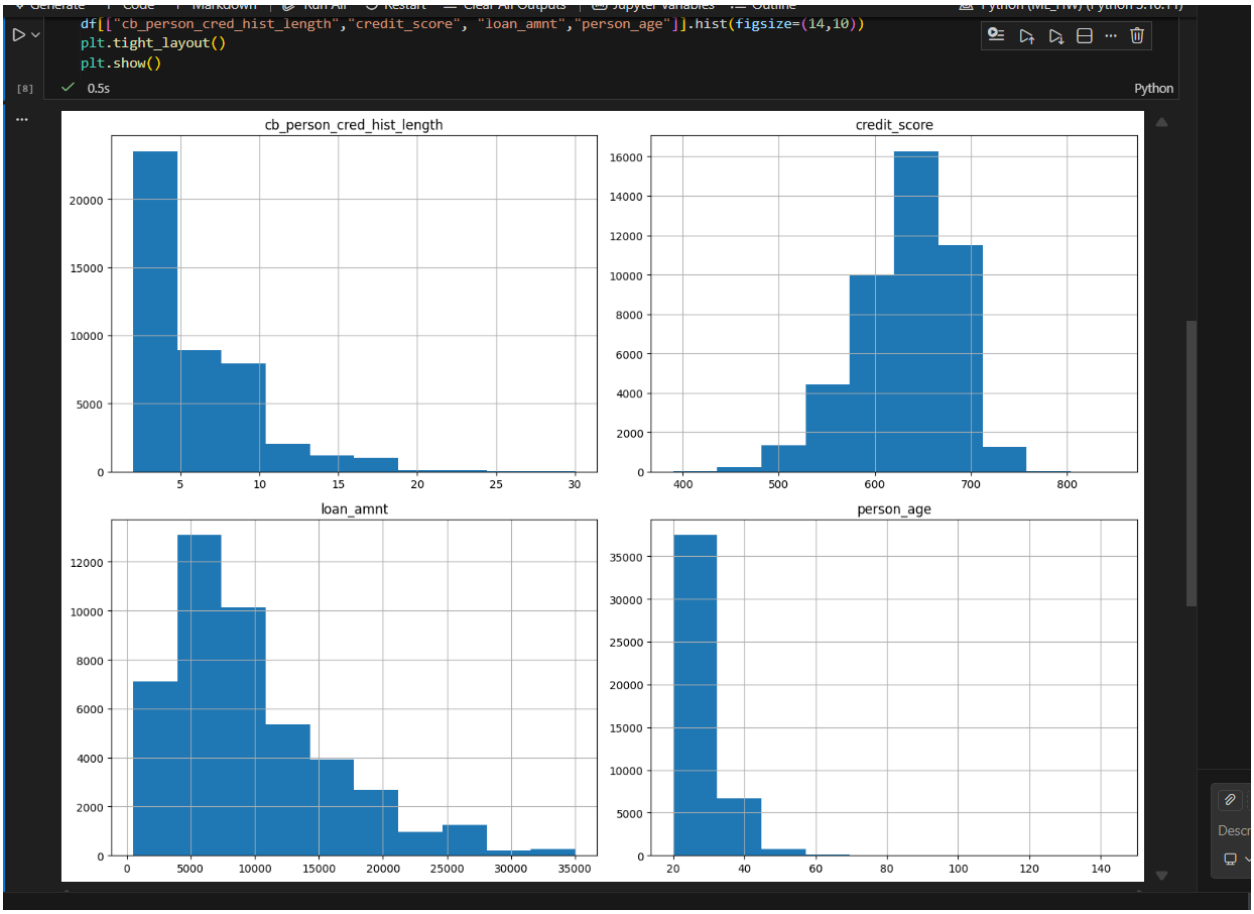
analytically, while stochastic gradient descent is an iterative optimization method that gradually approaches the minimum of the cost function.

Also, we used normalization for our stochastic gradient descent, because the noise was giving extremely large numbers as errors. The theta values are different, but the output is very close, while Normal equation gives the least amount of error, the difference in error was marginal.

2. 1. There are 13 input features:

'person_age', 'person_gender', 'person_education', 'person_income', 'person_emp_exp', 'person_home_ownership', 'loan_amnt', 'loan_intent', 'loan_int_rate', 'loan_percent_income', 'cb_person_cred_hist_length', 'credit_score', 'previous_loan_defaults_on_file'

2. 2. A)Histogram:



Histogram showing distribution of cb_person_cred_hist_length, credit_score, loan_amnt, person_age.

In cb_person_cred_hist_length, we see that most people have credit history length of (between 0-5 years). This suggests the dataset mainly consists of relatively new borrowers. We will need data scaling for this.

The credit_score shows an approximately normal (bell-shaped) distribution centered around the mid-600s. This indicates most borrowers have moderate creditworthiness

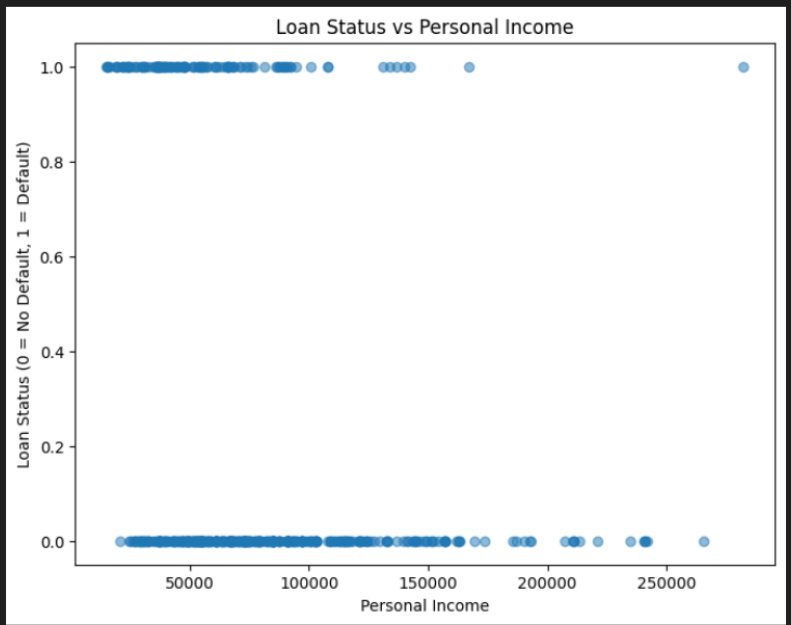
loan_amnt is has most distribution from 0-10000, hence most loans are relatively small.

Person_age is concentrated between approximately 20 and 40 years, showing a narrow distribution. Compared to loan amount, the scale of this feature is much smaller. We will need data scaling for this.

b) Randomly sample 500 data points , scatter plot of loan status with respect to personal income

```
sample_df = df.sample(500, random_state=42)
plt.figure(figsize=(8,6))
plt.scatter(sample_df["person_income"], sample_df["loan_status"], alpha=0.5)
plt.xlabel("Personal Income")
plt.ylabel("Loan Status (0 = No Default, 1 = Default)")
plt.title("Loan Status vs Personal Income")
plt.show()
```

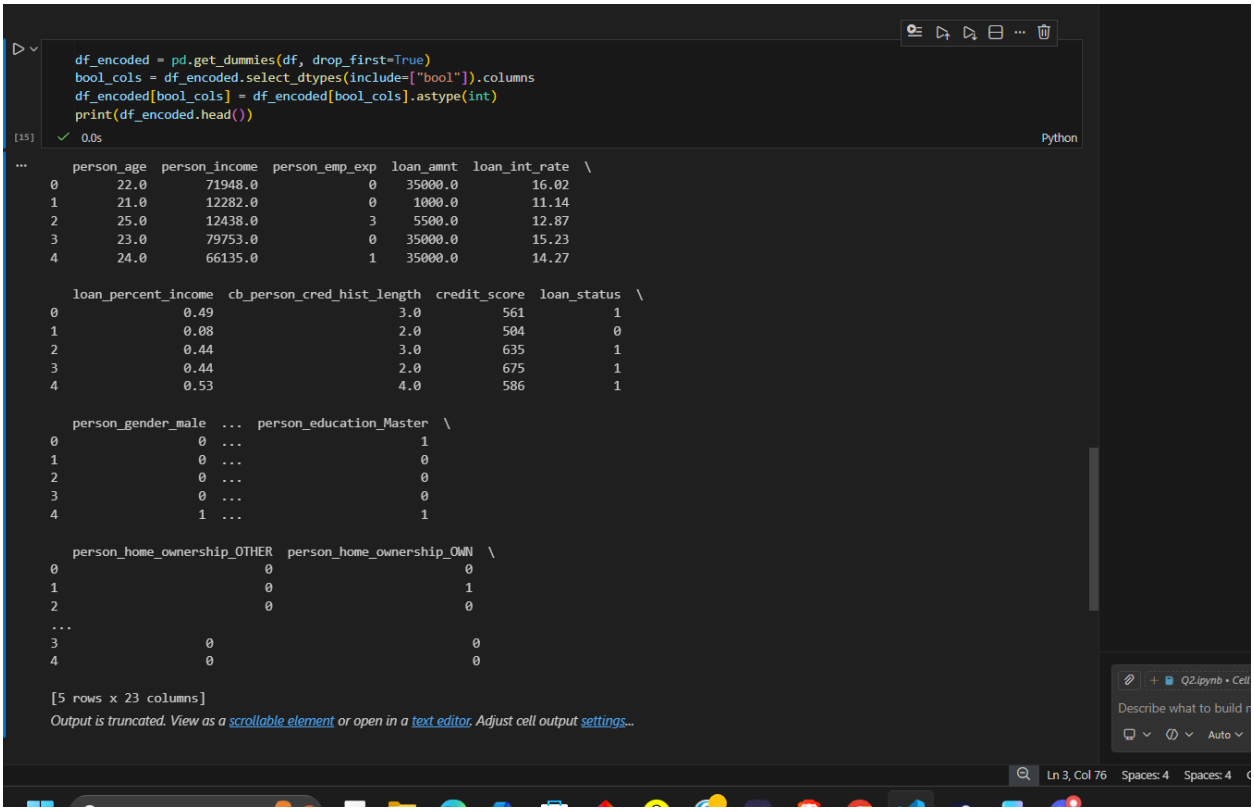
0.1s Python



At lower income levels, there is a mixture of defaults and non-defaults, indicating higher uncertainty in repayment ability. As personal income increases, the number of defaults

decreases significantly. High-income borrowers are mostly concentrated in the non-default category.

3. We will use one-hot encoding for person_gender, person_education, person_income, person_home_ownership, loan_intent, previous_loan_defaults_on_file.



```
df_encoded = pd.get_dummies(df, drop_first=True)
bool_cols = df_encoded.select_dtypes(include=["bool"]).columns
df_encoded[bool_cols] = df_encoded[bool_cols].astype(int)
print(df_encoded.head())
```

[15] ✓ 0.0s Python

```
   person_age  person_income  person_emp_exp  loan_amnt  loan_int_rate  \
0        22.0        71948.0               0    35000.0          16.02
1        21.0        12282.0               0     1000.0          11.14
2        25.0        12438.0               3     5500.0          12.87
3        23.0        79753.0               0    35000.0          15.23
4        24.0        66135.0               1    35000.0          14.27

   loan_percent_income  cb_person_cred_hist_length  credit_score  loan_status  \
0                 0.49                         3.0           561            1
1                 0.08                         2.0           504            0
2                 0.44                         3.0           635            1
3                 0.44                         2.0           675            1
4                 0.53                         4.0           586            1

   person_gender_male  ...  person_education_Master  \
0                   0  ...                        1
1                   0  ...                        0
2                   0  ...                        0
3                   0  ...                        0
4                   1  ...                        1

   person_home_ownership_OTHER  person_home_ownership_OWN  \
0                            0                          0
1                            0                          1
2                            0                          0
...
3                            0                          0
4                            0                          0

[5 rows x 23 columns]
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

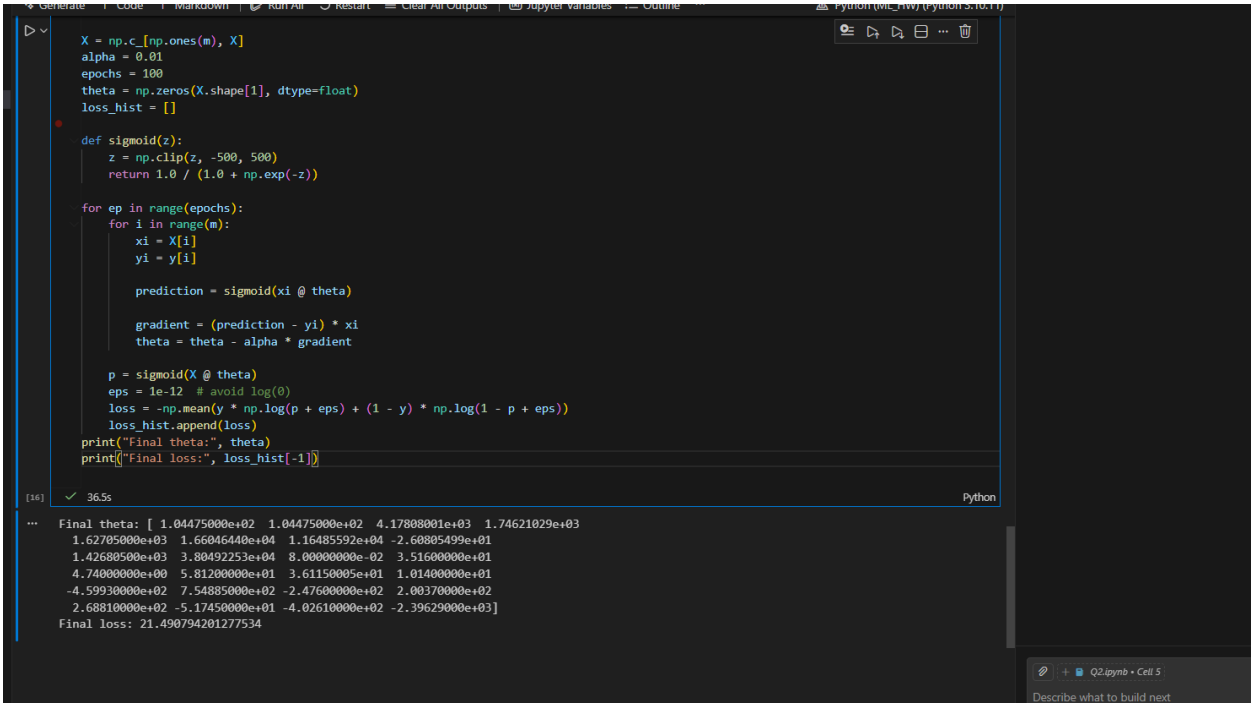
Q2.ipynb • Cell

Describe what to build n

Auto

Ln 3, Col 76 Spaces: 4 Spaces: 4

We compute the theta,



```
X = np.c_[np.ones(m), X]
alpha = 0.01
epochs = 100
theta = np.zeros(X.shape[1], dtype=float)
loss_hist = []

def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))

for ep in range(epochs):
    for i in range(m):
        xi = X[i]
        yi = y[i]

        prediction = sigmoid(xi @ theta)

        gradient = (prediction - yi) * xi
        theta = theta - alpha * gradient

    p = sigmoid(X @ theta)
    eps = 1e-12  # avoid log(0)
    loss = -np.mean(y * np.log(p + eps) + (1 - y) * np.log(1 - p + eps))
    loss_hist.append(loss)
print("Final theta:", theta)
print("Final loss:", loss_hist[-1])
```

[16] ✓ 36.5s Python

```
Final theta: [ 1.04475000e+02  1.04475000e+02  4.17808001e+03  1.74621029e+03
  1.62705000e+03  1.66046440e+04  1.16485592e+04 -2.60805499e+01
  1.42680500e+03  3.80492253e+04  8.00000000e-02  3.51600000e+01
  4.74000000e+00  5.81200000e+01  3.61150005e+01  1.01400000e+01
 -4.59930000e+02  7.54885000e+02 -2.47600000e+02  2.00370000e+02
  2.68810000e+02 -5.17450000e+01 -4.02610000e+02 -2.39629000e+03]
Final loss: 21.490794201277534
```

Q2.ipynb • Cell 5

Describe what to build next

Our theta is:

[1.04475000e+02 1.04475000e+02 4.17808001e+03 1.74621029e+03

  1.62705000e+03 1.66046440e+04 1.16485592e+04 -2.60805499e+01

  1.42680500e+03 3.80492253e+04 8.00000000e-02 3.51600000e+01

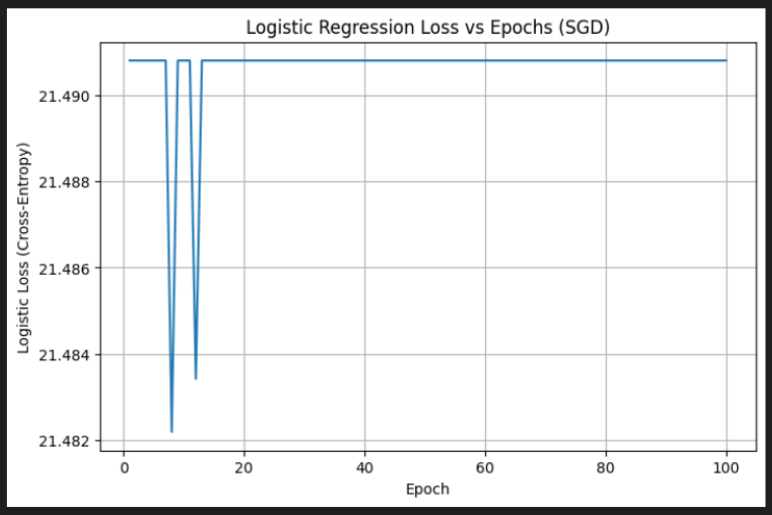
  4.74000000e+00 5.81200000e+01 3.61150005e+01 1.01400000e+01

 -4.59930000e+02 7.54885000e+02 -2.47600000e+02 2.00370000e+02

  2.68810000e+02 -5.17450000e+01 -4.02610000e+02 -2.39629000e+03]

```
plt.figure(figsize=(8,5))
plt.plot(range(1, epochs+1), loss_hist)
plt.xlabel("Epoch")
plt.ylabel("Logistic Loss (Cross-Entropy)")
plt.title("Logistic Regression Loss vs Epochs (SGD)")
plt.grid(True)
plt.show()
```

[17] ✓ 0.0s Python



Without normalization,

Now we will normalize the data,

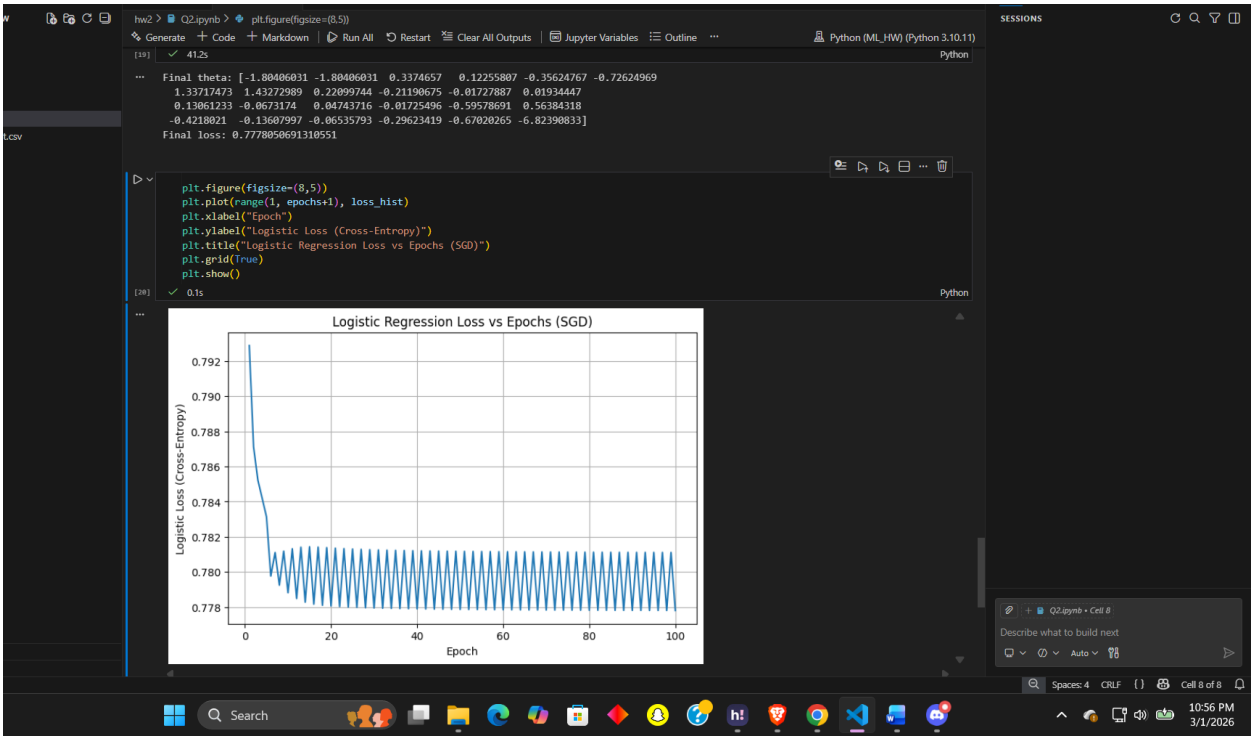
```
X = np.c_[np.ones(m), X]
alpha = 0.01
epochs = 100
theta = np.zeros(X.shape[1], dtype=float)
loss_hist = []

def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))

for ep in range(epochs):
    for i in range(m):
        xi = X[i]
        yi = y[i]

        prediction = sigmoid(xi @ theta)

        gradient = (prediction - yi) * xi
        theta = theta - alpha * gradient

    p = sigmoid(X @ theta)
    eps = 1e-12  # avoid log(0)
    loss = -np.mean(y * np.log(p + eps) + (1 - y) * np.log(1 - p + eps))
    loss_hist.append(loss)
print("Final theta:", theta)
print("Final loss:", loss_hist[-1])
```

[19] ✓ 41.2s Python

```
Final theta: [-1.80406031 -1.80406031  0.3374657   0.12255807 -0.35624767 -0.72624969
  1.33717473  1.43272989  0.22099744 -0.21190675 -0.01727887  0.01934447
  0.13061233 -0.0673174   0.04743716 -0.01725496 -0.59578691  0.56384318
 -0.4218021  -0.13607997 -0.06535793 -0.29623419 -0.67020265 -6.82390833]
Final loss: 0.7778050691310551
```

```
plt.figure(figsize=(8,5))
plt.plot(range(1, epochs+1), loss_hist)
plt.xlabel("Epoch")
```

[20] ✓ 0.1s Python



Our theta is: [-1.80406031 -1.80406031 0.3374657   0.12255807 -0.35624767 -0.72624969

1.33717473 1.43272989 0.22099744 -0.21190675 -0.01727887 0.01934447

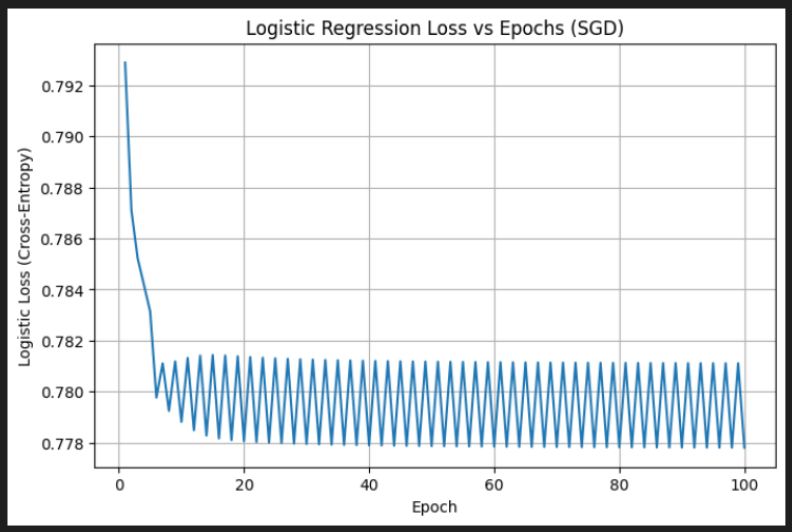
0.13061233 -0.0673174   0.04743716 -0.01725496 -0.59578691 0.56384318

-0.4218021 -0.13607997 -0.06535793 -0.29623419 -0.67020265 -6.82390833]

b)

```
plt.figure(figsize=(8,5))
plt.plot(range(1, epochs+1), loss_hist)
plt.xlabel("Epoch")
plt.ylabel("Logistic Loss (Cross-Entropy)")
plt.title("Logistic Regression Loss vs Epochs (SGD)")
plt.grid(True)
plt.show()
```

[20] ✓ 0.1s Python



c)

```
# accuracy
probs = sigmoid(X @ theta)              # predicted probabilities
y_pred = (probs >= 0.5).astype(int)     # predicted labels 0/1

# confusion pieces
TP = np.sum((y == 1) & (y_pred == 1))
TN = np.sum((y == 0) & (y_pred == 0))
FP = np.sum((y == 0) & (y_pred == 1))
FN = np.sum((y == 1) & (y_pred == 0))

accuracy = (TP + TN) / (TP + TN + FP + FN) * 100

print(f"TP={TP}, TN={TN}, FP={FP}, FN={FN}")
print(f"Accuracy on entire dataset: {accuracy:.2f}%")
```

[21] ✓ 0.0s Python

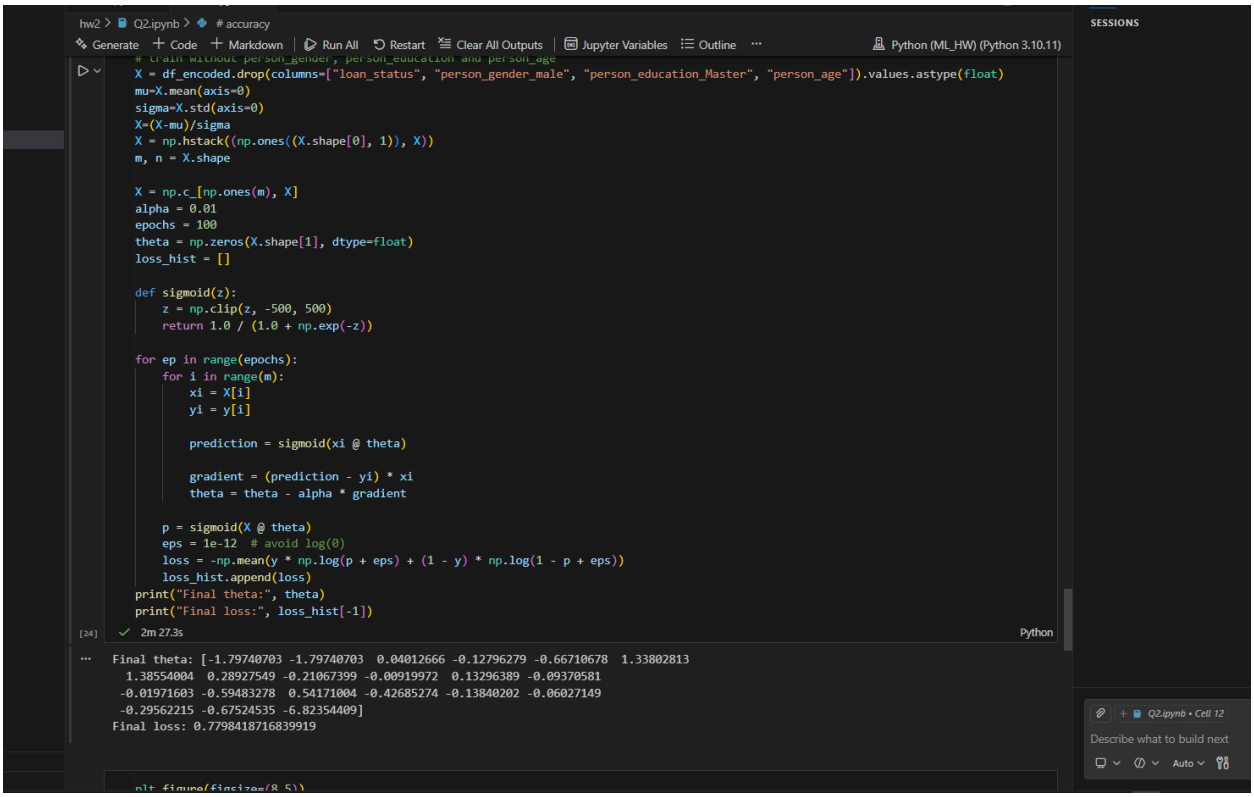
```
TP=9970, TN=23895, FP=11105, FN=30
Accuracy on entire dataset: 75.26%
```

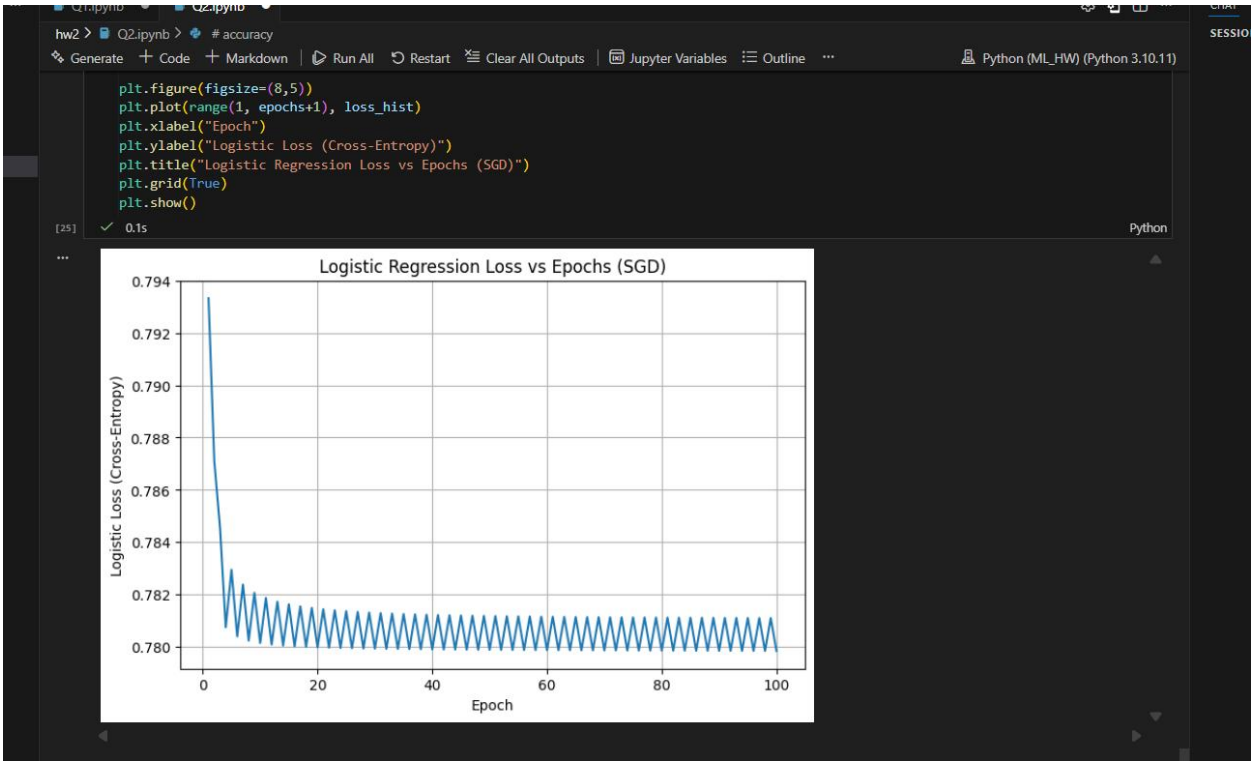
Q2.ipynb • Cell 9
Describe what to build next
Auto
Spaces: 4 CRLF { } Cell 9 of 9

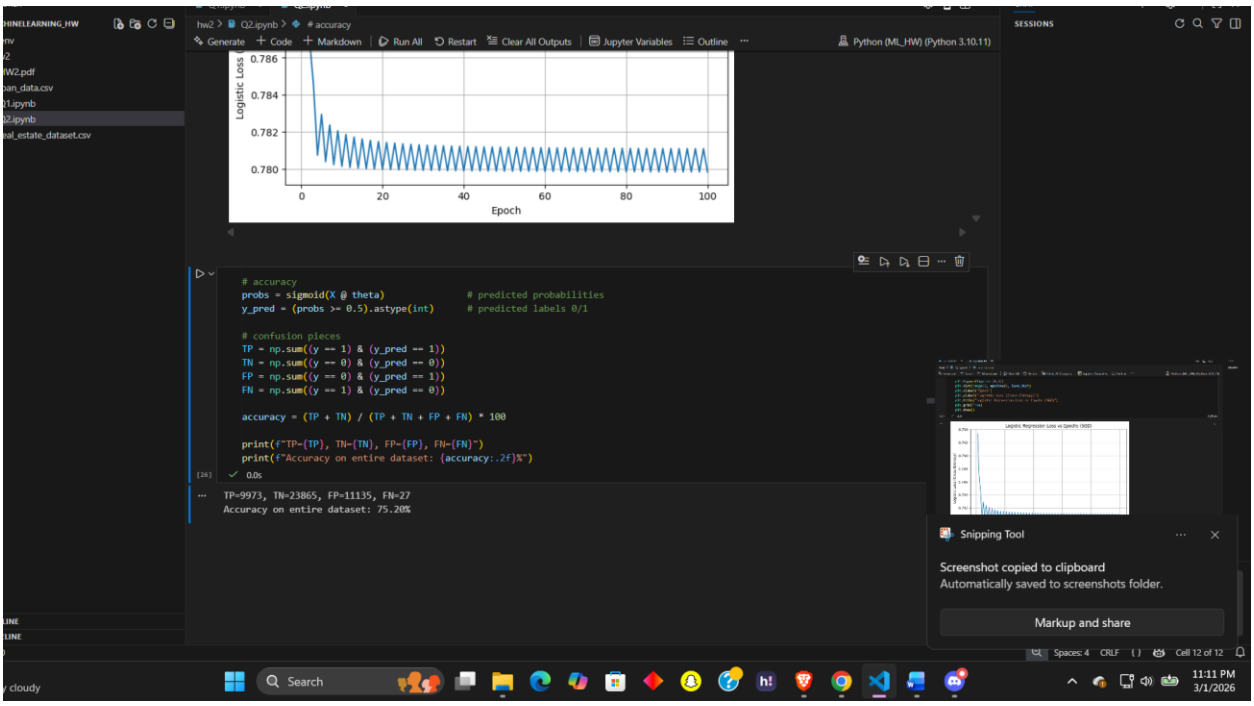
Accuracy: 75.26%

4. a) we can remove person_gender, person_education and person_age, as they should not affect loan as they are irrelevant to credit worthiness.

   b)







We were able to get 75.20 % accuracy.

c) Comparing the two cases, we see an accuracy percentage difference of 0.06 %, which is very less, so our assumption was correct.